Qt Simple Guide

QSizePolicy Examples

`QSizePolicy` tells a layout how a widget wants to grow or shrink. It has two axes (horizontal, vertical) set independently, plus a stretch factor.

The Policy Values Explained

```
QSizePolicy::Fixed        // stays exactly at sizeHint() — never grows or shrinks
QSizePolicy::Minimum      // can grow, but won't shrink below sizeHint()
QSizePolicy::Maximum      // can shrink, but won't grow beyond sizeHint()
QSizePolicy::Preferred    // sizeHint() is ideal, but can grow or shrink (default for most widgets)
QSizePolicy::Expanding    // actively wants extra space — will grab it from Preferred neighbours
QSizePolicy::MinimumExpanding // like Expanding but won't shrink below sizeHint()
QSizePolicy::Ignored      // sizeHint() is ignored entirely — takes whatever space is available
```

The key distinction to internalise: **Preferred** widgets *accept* extra space passively, while **Expanding** widgets *request* it actively. When both are in the same layout, `Expanding` wins the extra room.

Example 1 — Fixed vs Expanding

A label that stays a fixed size next to an input that takes all remaining width:

```
QHBoxLayout *layout = new QHBoxLayout;

QLabel *label = new QLabel("Username:");
label->setSizePolicy(QSizePolicy::Fixed, QSizePolicy::Fixed);
// equivalent shorthand — Fixed in both axes:
// label->setFixedWidth(80);

QLineEdit *input = new QLineEdit;
input->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Fixed);

layout->addWidget(label);
layout->addWidget(input);
```

Result: the label stays at its natural width; the input stretches to fill all remaining horizontal space.

Example 2 — Preferred vs Expanding (the subtle one)

```
QHBoxLayout *layout = new QHBoxLayout;

QPushButton *btnA = new QPushButton("Preferred");
btnA->setSizePolicy(QSizePolicy::Preferred, QSizePolicy::Fixed);

QPushButton *btnB = new QPushButton("Expanding");
btnB->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Fixed);

layout->addWidget(btnA);
layout->addWidget(btnB);
```

Result: `btnB` takes the lion's share of the space. `btnA` sits at its `sizeHint()` width while `btnB` absorbs everything left over. This is useful when you want one dominant widget alongside a naturally-sized companion.

Example 3 — Stretch Factor to Split Space

When multiple `Expanding` widgets compete, stretch factors decide the ratio:

```
QHBoxLayout *layout = new QHBoxLayout;

QTextEdit *editor = new QTextEdit;
editor->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);

QTextEdit *preview = new QTextEdit;
preview->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);

// Editor gets 2/3 of the width, preview gets 1/3
layout->addWidget(editor,  2);
layout->addWidget(preview, 1);
```

This is how you'd build a side-by-side editor/preview pane — the ratio holds when the window is resized.

Example 4 — Vertical Policies in a VBoxLayout

```
QVBoxLayout *layout = new QVBoxLayout;

QLabel *title = new QLabel("Report");
title->setSizePolicy(QSizePolicy::Preferred, QSizePolicy::Fixed);
// Fixed vertically — won't grow taller than its text

QTextEdit *body = new QTextEdit;
body->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
// Expands to fill all remaining vertical space

QLabel *footer = new QLabel("Page 1 of 1");
footer->setSizePolicy(QSizePolicy::Preferred, QSizePolicy::Fixed);

layout->addWidget(title);
layout->addWidget(body);
layout->addWidget(footer);
```

Result: the title and footer stay slim; the text editor fills all vertical space between them.

Example 5 — Maximum Policy (capping growth)

```
QVBoxLayout *layout = new QVBoxLayout;

QLineEdit *searchBox = new QLineEdit;
searchBox->setSizePolicy(QSizePolicy::Maximum, QSizePolicy::Fixed);
searchBox->setMaximumWidth(300);
// Won't grow beyond 300px even in a wide window

QTextEdit *results = new QTextEdit;
results->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);

layout->addWidget(searchBox);
layout->addWidget(results);
```

Example 6 — Ignored Policy (spacer-like widget)

```
// A custom divider widget that takes up all available space
// regardless of what sizeHint() returns
QFrame *spacer = new QFrame;
spacer->setSizePolicy(QSizePolicy::Ignored, QSizePolicy::Ignored);
spacer->setFrameShape(QFrame::HLine);

layout->addWidget(spacer);
```

`Ignored` is rarely needed day-to-day — `QSpacerItem` or `addStretch()` are usually better for blank space — but it's useful for custom widgets that should be completely layout-neutral.

Example 7 — Setting Both Axes at Once with the Constructor

Instead of calling `setSizePolicy()` with two separate enum values, you can use the `QSizePolicy` constructor directly when you want to also set the stretch or other flags:

```
// Constructor: QSizePolicy(horizontal, vertical)
QSizePolicy policy(QSizePolicy::Expanding, QSizePolicy::Fixed);
policy.setHorizontalStretch(2);   // this widget gets 2x horizontal stretch
policy.setRetainSizeWhenHidden(true);  // keeps its space even when hidden

widget->setSizePolicy(policy);
```

`setRetainSizeWhenHidden(true)` is particularly handy for toolbar buttons or status indicators that toggle visibility — without it, hiding a widget causes the layout to reflow and shift everything else around.

Quick Reference Card

You want...	Policy to use
Fixed-size widget (never resizes)	`Fixed` / `Fixed`
Label beside an input field	`Fixed` / `Fixed` label, `Expanding` / `Fixed` input
Full-area canvas or editor	`Expanding` / `Expanding`
Button that doesn't stretch	`Minimum` / `Fixed`
Sidebar with max width	`Maximum` / `Expanding`
Two panes split at 2:1 ratio	`Expanding` both, stretch factors 2 and 1
Widget that holds space when hidden	`setRetainSizeWhenHidden(true)`